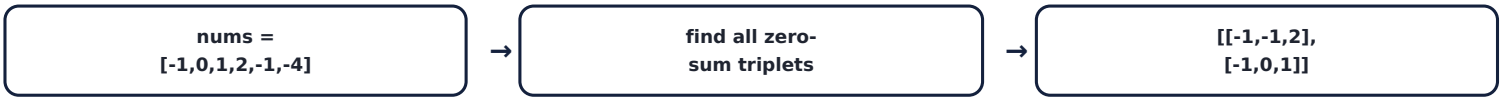
3Sum

Two approaches, visualized execution, and the core logic to remember

Brute Force · $O(n^3)$ Two Pointers · $O(n^2)$

Problem Statement

Given an integer array `nums`, return all unique triplets `[nums[i], nums[j], nums[k]]` where `i != j != k` and the three values sum to zero. No duplicate triplets in the result.



Approach Comparison

Approach	Time	Space	Core Idea
Brute Force	$O(n^3)$	O(1)*	Sort, then check every triplet with dedup guards
Two Pointers	$O(n^2)$	O(1)*	Fix one number, two-pointer the rest of the sorted array

Big picture: 3Sum is 2Sum with an extra outer loop. Fixing `nums[i]` turns "find a triplet summing to zero" into "find a pair summing to `-nums[i]`" — the exact sorted two-pointer pattern from 2Sum, just run once per `i`.

1 · Brute Force

O(n³) time **O(1) extra space***

Sort array → **Try every (i, j, k)** → **Skip repeated values per level** → **sum == 0? collect triplet**

```java
// sort first so duplicates sit next to each other, then brute-force every triplet
private static List<List<Integer>> bruteforceSolution(int[] nums) {
    List<List<Integer>> output = new ArrayList<>();
    Arrays.sort(nums);

    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) { continue; }

        for (int j = i + 1; j < nums.length - 1; j++) {
            if (j > i + 1 && nums[j] == nums[j - 1]) { continue; }

            for (int k = j + 1; k < nums.length; k++) {
                if (k > j + 1 && nums[k] == nums[k - 1]) { continue; }

                int sum = nums[i] + nums[j] + nums[k];
                if (sum == 0) {
                    output.add(Arrays.asList(nums[i], nums[j], nums[k]));
                }
            }
        }
    }
    return output;
}
```

Recall trick: "Sort first so duplicates sit next to each other, then brute-force every triplet while skipping repeats at each level." Simple and correct, but O(n³) — too slow for large inputs.

2 · Two Pointers

O(n²) time **O(1) extra space***

Sort array, fix nums[i] → **left = i+1 right = n-1** → **sum vs 0: move left/right** → **match → collect, skip dup values**

```java
// fix one number, then run the classic sorted two-pointer pair-sum on the rest
private static List<List<Integer>> usingTwoPointerSolution(int[] nums) {
    List<List<Integer>> output = new ArrayList<>();
    Arrays.sort(nums);

    for (int i = 0; i < nums.length - 2; i++) {
        int left = i + 1;
        int right = nums.length - 1;

        if (i > 0 && nums[i] == nums[i - 1]) { continue; }

        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                output.add(Arrays.asList(nums[i], nums[left], nums[right]));
                left++;
                right--;

                while (left < nums.length && nums[left] == nums[left - 1]) { left++; }
                while (right >= 0 && nums[right] == nums[right + 1]) { right--; }
            } else if (sum < 0) {
                left++;
            } else {
                right--;
            }
        }
    }
    return output;
}
```

Trace for `nums = [-1,0,1,2,-1,-4]` → sorted `[-4,-1,-1,0,1,2]`:

i	nums[i]	left/right walk	found
0	-4	all sums < 0 as left advances to meet right	none
1	**-1**	**(2,5): -1-1+2=0 ✓ | (3,4): -1+0+1=0 ✓**	**[-1,-1,2] & [-1,0,1]**
2	-1	nums[2]==nums[1] → skipped (duplicate i)	—
3	0	(4,5): 0+1+2=3>0 → right-- → left<right false	none

Recall trick: "Fix one number, two-pointer the rest." For each `i`, the problem collapses into "find two numbers in the remaining sorted range that sum to `-nums[i]`" — exactly the sorted two-pointer pattern, nested inside a loop over `i`. Duplicate triplets are skipped by comparing each pointer to its previous position, since sorting keeps identical values adjacent.

TL;DR — For Future Me

What to remember

- **Best solution:** Two pointers, O(n²) — "fix one number, then run the classic two-pointer pair-sum on what's left."
- **Brute force** is O(n³) — correct but only fine for small inputs.
- **Underlying pattern:** 3Sum is 2Sum with an extra outer loop. Any "kSum" problem generalizes this way — fix `k-2` numbers with nested loops, then two-pointer the final two.
- **Watch for duplicates:** sorting first makes identical values sit next to each other, so a simple `== previous` check at each pointer level is enough to dedup — no `Set` needed.